Imagine needing to engage in multiple interactions with LLMs before arriving at a final output. Perhaps we want to utilize various tools and combine their responses into a structured format. This need arises because LLM responses are often not in the desired format or may lack the accuracy required for real-life applications.

In many cases, we find ourselves needing to implement quality control steps that could involve reengaging with the LLMs to refine or validate the information provided. This process of "chaining" calls—where multiple interactions with LLMs and other tools are orchestrated—is essential for creating robust and reliable AI-driven solutions.

Chaining these steps together allows you to build more powerful and versatile AI applications that can handle complex tasks beyond simple prompt–response interactions. Implementing the necessary library code that performs all those tasks is a daunting process, but luckily for us, there is a specialized framework for this purpose called LangChain.

In this chapter, we'll work with the basic features of LangChain, including helpers for prompt management, memory, agents, content ingestion, and retrieval. Then we'll apply this knowledge by integrating LangChain's capabilities into our app and using it alongside the Vercel AI SDK.

6.1 Introducing LangChain

LangChain (https://js.langchain.com/) is a tool/framework dedicated to providing abstractions that make it easier to chain multiple steps together into a cohesive workflow. The core concept in LangChain for this purpose is the chain, or what is more technically a series of linked function call that, together, provide functionality. LangChain deals with the underlying technicalities of managing the API interactions, parsing and transforming content and interacting with various database stores and providers while exposing a composable API for developers to use.

The LangChain ecosystem of tools is huge, and exploring all of its available features would require us to write another book entirely. Instead, we focus on the main concepts that will allow us to combine its features with our existing web application. Our main goal here is to understand how things work together and how to integrate each relevant feature in the most suitable way.

The LangChain project library is provided in two programming languages: one with Python and one in JavaScript, which is called LangChain.js. The LangChain.js project is organized into several modules, each serving a specific purpose in the development of LLM-powered applications. Table 6.1 provides a quick overview of the main modules.

Table 6.1 The main modules available from the LangChain.js library

Module	Description
`langchain/prompts`	Manages prompt templates and input variables
`langchain/agents`	Implements autonomous agents that can use tools and make decisions